ECE 7670
Program 7
Nathan Walker

November 4, 2025

---

1. for the field GF (2 4) generated by $1+x+x^4$ and show that the minimal polynomials of alpha, alpha 3, and alpha 5.

2. show that the generator for a (15,5) three-error-correcting binary BCH code is g(x) $= 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$.

3. determine the actual minimum weight of this code

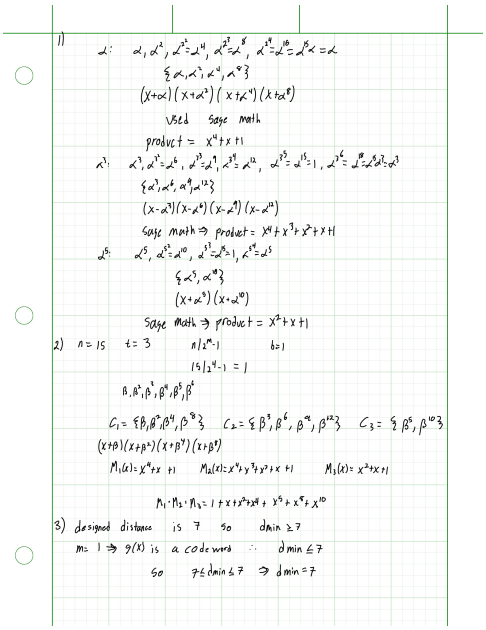


Figure 1: Hand work of the preliminary exercises

4. write a class ChienSearch that implements the Chien search algorithm. verify for the polynomial 1 + x + alpha$^5 * x^3 therootsarealpha^3, alpha^1 0, alpha^1 2$.

5. build a BCH decoder that decodes a vector of n binary elements.

6. thoroughly test your decoder testing all variations of 3 errors.

7. use the reduced complexity berlekamp-massey algorithm and repeat the tests.

8. use the sugiyama algorithm to decode the BCH.

```
//  Program: ChienSearch.cc
//  Todd K. Moon

#include "ChienSearch.h"
#include <vector>

GFNUM2m *
ChienSearch::Search(GFNUM2m *poly, int nu, int & nroots)
// Lambda(x) = poly[0] + poly[1] x + ... + poly[nu]x^nu
{
    nroots = 0;
    GFNUM2m* roots = new GFNUM2m[16];
    for(int i = 0; i < 15; i++)
    {
        GFNUM2m evaluation = A^i;
        GFNUM2m sum = GFNUM2m(0);
        for(int j = 0; j <= nu ; j++)
        {
            GFNUM2m temp_evaluation = evaluation;
            temp_evaluation ^= j;
            sum += temp_evaluation * poly[j];
        }
        if(sum == 0)
        {
            //cout << evaluation << endl<<endl;
            roots[nroots++] = evaluation;
        }
    }
    // setnroots(nroots);
   // GFNUM2m *roots = v_roots.data();
    //nroots -= 1;
   return roots;
}



/*
Local Variables:
compile-command: "g++ -c -g ChienSearch.cc"
End:
*/
```

Listing 1: ChienSearch.cc

```
// BCHdec.cc -- a general BCH decoder
// Todd K. Moon
#include "BCHdec.h"
```

```
#include "berlmass.cc"
//#include "berlmassBCH2.cc"

//#include "berlmassBCH.cc"
#include "polynomialT.cc"

#include "gcd_poly.cc"

BCHdec::BCHdec(int t_in, int n_in, GFNUM2m A1_in) :
   Searcher(t_in)
{
   t = t_in;
   t2 = 2*t_in;
   n = n_in;
   A1 = A1_in;
   s = new GFNUM2m[2*t];
   Lambda = new GFNUM2m[t+1];
}

void
BCHdec::decode(GF2 *r, GF2 *dec)
{
   int i,j;
   int errloc;
   GFNUM2m p;
   GFNUM2m x;
   GFNUM2m *rs;                  // array of roots

   // Step 1: evaluate the syndromes

    //cout << endl << "computing the syndromes" << endl;
   for(i = 1; i <= t2; i++){
        GFNUM2m sum = GFNUM2m(0);
        GFNUM2m evaluating_term = A^i;
        sum += r[0];
        for(j = 1; j < 15; j++){
            if(r[j] == GF2(1)){
                sum += evaluating_term;
            }
            evaluating_term *= A^i;
        }
        s[i-1] = sum;
        //cout << s[i-1] << " ";
    }
    //cout << endl;


   // Step 2: Determine the error locator polynomial
    //cout << "computing the error locator polynomial" << endl;
    GFNUM2m incoeffx[7] = {GFNUM2m(0),GFNUM2m(0),GFNUM2m(0),GFNUM2m
       (0)
```

```
    ,GFNUM2m(0),GFNUM2m(0),GFNUM2m(1)};
    polynomialT<GFNUM2m> x_temp(6, incoeffx);
    GFNUM2m syndromecoeff[6] = {s[0], s[1], s[2], s[3], s[4], s[5]};
    polynomialT<GFNUM2m> syndrome(5, syndromecoeff);
    polynomialT<GFNUM2m> gsugi, ssugi, tsugi;
    int psugi;
    sugiyama(x_temp,syndrome,gsugi,ssugi,tsugi,psugi);
    //polynomialT<GFNUM2m> c = berlmass(s, t2);
    //polynomialT<GFNUM2m> c = berlmassBCH(s, t2);
    int deg = tsugi.getdegree();
    for(i = 0; i < t+1; i++){
        Lambda[i] = GFNUM2m(0);
        if(i <= deg){
            Lambda[i] = tsugi[i];
        }
        //cout << Lambda[i] << " ";
    }
    //cout << endl;
   // (or berlmass or berlmass2...)

   // Step 3: Find the roots of the error locator polynomial
   int nroots;
   rs = Searcher.Search(Lambda,nu,nroots); // from ChienSearch

   // Step 4: Find error values: Not necessary for binary BCH codes

   // Step 5: Correct errors corresponding to inverse roots
   for(i = 0; i < n; i++) {  // copy over the input bits
      dec[i] = r[i];

   }
    if(nroots == 0)
        nroots++;
    //cout << "nroots = "<< nroots << endl;
    //cout << "error locations" << endl;
   for(i = 0; i < nroots; i++) {
      errloc = (GFNUM2m::getN() - rs[i].getp()) % GFNUM2m::getN();
      //cout << errloc << endl;
      dec[errloc] = 1+dec[errloc];  // complement the bits
   }
}


/*
Local Variables:
compile-command: "g++ -c -g BCHdec.cc"
End:
*/
```

Listing 2: BCHdec.cc